运用风险指针实现无锁栈

Posted on 2023-12-16 17:23:31 by 恋恋风辰 | In C++ | 原文链接 (/articlepage?id=2ZcPttacvKKxqsv6DBhnsBjY4Xy)

简介

术语“风险指针”是指Maged Michael发明的一种技法， 后来被IBM申请为专利。
前文我们设计了无锁并发栈的结构，对于pop操作回收节点采用的是延时删除的策略，即将要删除的节点放入待删除列表中。
但是待删列表中的节点可能永远不会被回收，因为每次多个线程pop就不会触发回收待删列表的操作。上一节我们说可以通过执行pop的最后一个线程执行回收，那为了实现这个目的，我们就要换一种思路。就是我们将要删除的节点做特殊处理，如果有线程使用它，就将他标记为正在使用，那么这个节点的指针就是风险指针，也就是不能被其他线程删除。

改进pop

我们假设有一块全局区域存储的是一个风险数组，数组里放的就是风险指针。
我们提供一个函数去设置数组中某个节点为风险指针，没被设置为风险指针的节点就是可用的。
然后我们再提供一个函数去查找数组中的节点，返回一个可用的节点。
假设查找的函数叫做 `get_hazard_pointer_for_current_thread` 。

```
1. std::shared_ptr<T> pop()
2. {
3.     // 1
4.     std::atomic<void*>& hp=get_hazard_pointer_for_current_thread();
5.     node* old_head=head.load();  // <---- 2
6.     node* temp;
7.     do
8.     {
9.         temp=old_head;
10.        hp.store(old_head);  // <---- 3
11.        old_head=head.load();
12.    } while(old_head!=temp);  // <---- 4
13.    // ...
14. }
```

1 处获取数组中可用节点指针，用hp存储节点指针的引用。

2 处加载当前头节点保存到old节点里。

3 处将old_head的值赋值给hp。

4 while循环处的作用就是防止多个线程访问pop函数，某一个线程B将head修改，那说明他已经将它的临时变量old节点放入风险数组中，而本线程A的old节点和线程B的old节点指向的是同一个旧有的head，所以线程A就没必要将这个old节点放入风险数组了，需要再次循环获取新的head加载为old节点，再放入风险数组。

我们实现一个完整意义的pop操作

```
1. std::shared_ptr<T> pop()
2. {
3.     //1 从风险列表中获取一个节点给当前线程
4.     std::atomic<void*>& hp=get_hazard_pointer_for_current_thread();
5.     node* old_head=head.load();
6.     do
7.     {
8.         node* temp;
9.         do
10.        {
11.            temp=old_head;
12.            hp.store(old_head);
13.            old_head=head.load();
14.        }//2 如果old_head和temp不等说明head被其他线程更新了，需重试
15.        while(old_head!=temp);
16.    }//3 将当前head更新为old_head->next，如不满足则重试
17.    while(old_head&&
18.          !head.compare_exchange_strong(old_head,old_head->next));
19.    // 4一旦更新了head指针，便将风险指针清零
20.    hp.store(nullptr);
21.    std::shared_ptr<T> res;
22.    if(old_head)
23.    {
24.        res.swap(old_head->data);
25.        //5 删除旧有的头节点之前，先核查它是否正被风险指针所指涉
26.        if(outstanding_hazard_pointers_for(old_head))
27.        {
28.            //6 延迟删除
29.            reclaim_later(old_head);
30.        }
31.        else
32.        {
33.            //7 删除头部节点
34.            delete old_head;
35.        }
36.        //8 删除没有风险的节点
37.        delete_nodes_with_no_hazards();
38.    }
39.    return res;
40. }
```

1 我们观察1处代码从全局的风险数组中jiu获取一个可用的节点作为风险节点。这样其他线程

在回收节点的时候会从这个数组中看到这个风险节点，进而不会删除该节点。

2 处代码做循环比较是为了hp存储的节点为当前线程操作的head，如果其它线程更新了head，那么当前线程就要进行重试。以保证每个线程的hp存储的都是自己看到的最新的head。

3 处将head和old_head做比较，如果相等则更新head为old_head->next的值。如果不等则再次重复循环，因为有可能有多个线程都满足2处的条件，多个线程的hp和old_head指向相同，所以要重试，保证多线程情况下head的移动唯一。

4 一旦更新了head指针，就可以将这个风险指针清零了，因为其他线程pop操作的head已经不是我们hp存储的old_head了。所以此种情况下是线程安全的。

5 删除旧节点之前，先看它是否被风险指针所指涉。

6 如果要删除的节点被风险指针指涉，则延迟删除，放入待删列表。都则直接删除该节点即可。(7处)

8 每个线程pop后，都要查一下待删列表，将其中没有风险的节点删除。

接下来我们实现从全局连表中返回一个可用的节点

```
1. std::atomic<void*>& get_hazard_pointer_for_current_thread() {
2.     //每个线程都具有自己的风险指针 线程本地变量
3.     thread_local static hp_owner hazzard;
4.     return hazzard.get_pointer();
5. }
```

我们通过一个线程本地变量hazzard存储当前线程正在使用的节点，这个节点被称作风险节点。其他线程不能删除。

接下来我们实现hazard_pointer类，管理风险指针和线程id

```
1. struct hazard_pointer {
2.     std::atomic<std::thread::id> id;
3.     std::atomic<void*>  pointer;
4. };
```

id为正在使用该风险指针的id，pointer为指针类型，存储的节点数据地址。
当一个线程从风险数组中查找某个闲置节点作为风险节点，则需要将pointer指向节点的数据，并且将id设置为当前的线程id。

我们定义一个全局的风险节点数组，用来存储风险节点。

```
hazard_pointer hazard_pointers[max_hazard_pointers];
```

然后我们用hp_owner类管理这个风险指针

```
class hp_owner {
public:
    hp_owner(hp_owner const&) = delete;
    hp_owner operator=(hp_owner const&) = delete;
    hp_owner():hp(nullptr){
        for (unsigned i = 0; i < max_hazard_pointers; ++i) {
            std::thread::id  old_id;
            if (hazard_pointers[i].id.compare_exchange_strong(old_id, std::this_thread::get_id()))
    {
                hp = &hazard_pointers[i];
                break;
            }
        }

        if (!hp) {
            throw std::runtime_error("No hazard pointers available");
        }
    }

    ~hp_owner() {
        hp->pointer.store(nullptr);
        hp->id.store(std::thread::id());
    }
private:
    hazard_pointer* hp;
};
```

每个线程每次调用get_hazard_pointer_for_current_thread只会在第一次的时候构造hp_owner类型的hazzard，之后该线程再次调用该函数不会构造hp_owner，因为是线程本地变量。

当一个线程析构的时候会释放其本地变量hazzard，进而执行hp_owner析构函数，从而恢复初值。

接下来我们要实现hp_owner的get_pointer函数。

```
std::atomic<void*>& get_pointer() {
    return hp->pointer;
}
```

hp_owner 的get_pointer函数返回其成员pointer指向的地址

接下来我们实现判断该节点是否被风险指针所指涉的函数

```
    bool outstanding_hazard_pointers_for(void* p)
    {
        for (unsigned i = 0; i < max_hazard_pointers; ++i)
        {
            if (hazard_pointers[i].pointer.load() == p)
            {
                return true;
            }
        }
        return false;
    }
```

如果当前节点被风险指针所指涉则将该节点放入待删队列延迟删除

```
void reclaim_later(node* old_head) {
    add_to_reclaim_list(new data_to_reclaim(old_head));
}
```

将节点放入待删列表，我们封装了一个data_to_reclaim类型的节点放入待删列表。

我们定义待删节点的结构体

```
//待删节点
struct data_to_reclaim {
    node* data;
    std::function<void(node*)> deleter;
    data_to_reclaim* next;
    data_to_reclaim(node * p):data(p), next(nullptr){}
    ~data_to_reclaim() {
        delete data;
    }
};
```

然后在无锁栈中定义一个节点表示待删列表的首节点，因为栈是被多个线程操作的，待删列表也会被多个线程访问，那么我们需要用原子变量表示这个首节点

```
1. std::atomic<data_to_reclaim*>  nodes_to_reclaim;
```

我们实现将节点放入待删列表的逻辑

```
1. void add_to_reclaim_list(data_to_reclaim* reclaim_node) {
2.     reclaim_node->next = nodes_to_reclaim.load();
3.     while (!nodes_to_reclaim.compare_exchange_weak(reclaim_node->next, reclaim_node));
4. }
```

因为可能有多个线程同时将节点放入待删列表，所以此处做重试。
接下来实现从待删列表中删除无风险的节点

```
1. void delete_nodes_with_no_hazards() {
2.     data_to_reclaim* current = nodes_to_reclaim.exchange(nullptr);
3.     while (current) {
4.         data_to_reclaim* const next = current->next;
5.         if (!outstanding_hazard_pointers_for(current->data)) {
6.                 delete current;
7.         }
8.         else {
9.             add_to_reclaim_list(current);
10.        }
11.        current = next;
12.    }
13. }
```

测试

我们依然用之前的方式测试，启动三个线程t1用来向栈中放入数据(总计push20000个), t2和t3用来从栈中pop数据。用一个set记录删除的值，最后判断总共删除的数量是否为2000.

```
1. void TestHazardPointer() {
2.     hazard_pointer_stack<int> hazard_stack;
3.     std::set<int>  rmv_set;
4.     std::mutex set_mtx;
5.
6.     std::thread t1([&]() {
7.         for (int i = 0; i < 20000; i++) {
8.             hazard_stack.push(i);
9.             std::cout << "push data " << i << " success!" << std::endl;
10.            std::this_thread::sleep_for(std::chrono::milliseconds(5));
11.        }
12.        });
13.
14.    std::thread t2([&]() {
15.        for (int i = 0; i < 10000;) {
16.            auto head = hazard_stack.pop();
17.            if (!head) {
18.                std::this_thread::sleep_for(std::chrono::milliseconds(10));
19.                continue;
20.            }
21.            std::lock_guard<std::mutex> lock(set_mtx);
22.            rmv_set.insert(*head);
23.            std::cout << "pop data " << *head << " success!" << std::endl;
24.            i++;
25.        }
26.        });
27.
28.    std::thread t3([&]() {
29.        for (int i = 0; i < 10000;) {
30.            auto head = hazard_stack.pop();
31.            if (!head) {
32.                std::this_thread::sleep_for(std::chrono::milliseconds(10));
33.                continue;
34.            }
35.            std::lock_guard<std::mutex> lock(set_mtx);
36.            rmv_set.insert(*head);
37.            std::cout << "pop data " << *head << " success!" << std::endl;
38.            i++;
39.        }
40.        });
41.
42.    t1.join();
```

```
43.     t2.join();
44.     t3.join();
45.
46.     assert(rmv_set.size() == 20000);
47. }
```

优劣分析

风险指针的机制能保证要删除的节点在合理的时机回收，但是也引发了一些性能问题，比如为了删除某个节点要遍历风险数组判断该节点是否被风险指针所指涉。其次我们对于要删除的节点需要从风险数组中选择一个合适的节点记录其地址，所以也需要便利。
C++ 并发编程一书中提出了用空间换取时间和性能的办法，就是开辟2*N个大小的风险数组，只有当使用的节点达到N个时我们才依次判断N个节点是否被风险指针所指涉，这样我i们减少了判断回收的次数。但同样增加内存的开销。
另外作者也提及了风险指针为IBM的技术专利，即使我们懂得这种方法也不见得有权利使用，为后文提及了引用计数做了铺垫。

源码链接:

https://gitee.com/secondtonone1/boostasio-learn/tree/master/concurrent/day17-LockFreeStack (https://gitee.com/secondtonone1/boostasio-learn/tree/master/concurrent/day17-LockFreeStack)

视频链接:

https://space.bilibili.com/271469206/channel/collectiondetail?sid=1623290 (https://space.bilibili.com/271469206/channel/collectiondetail?sid=1623290)

浏览(1471) (JavaScript:void(0);) 评论(0) (JavaScript:void(0);) ♡喜欢(512) (JavaScript:void(0);)

热门评论

H B I

请输入正文

评论

没有更多评论了,点击收起 ≪ (JavaScript:void(0);)

热门文章

slice介绍和使用

♡喜欢(521) 浏览(1546)

C++ 类的继承封装和多态

♡喜欢(588) 浏览(2196)

windows环境搭建和vscode配置

♡喜欢(587) 浏览(1595)

解密定时器的实现细节

♡喜欢(566) 浏览(1449)

Linux环境搭建和编码

♡喜欢(594) 浏览(5076)

最新评论

C++ 类的继承封装和多态

gzy:谢谢老师，写的真的很好，本来对这些新的关键字都很模糊，几篇下来配合着ai记着笔记一下子就清晰了

C++ 右值引用与移动构造函数

李程浩:1

C++ 线程管控

secondtonone1:好的，已修复，写的时候马虎了，谢谢指正。

线程基础

secondtonone1:谢谢，笔误已修复

再谈单例模式

secondtonone1:是的，C++11以后返回局部static变量对象能保证线程安全了。

线程安全的无锁栈

secondtonone1:谢谢支持，如果pop的次数大于push的次数是会让线程处于重试的，这个是测试用例，必须满足push和pop的次数相同，实际情况不会这么使用。栈的设计没有问题。

创建项目和编译

secondtonone1:谢谢支持

visual studio配置boost库

secondtonone1:是的，您说的是对的，windows平台要采用msvc方式。

C++ 类的拷贝构造、赋值运算、单例模式

secondtonone1:好的，已修复。

asio多线程模式IOThreadPool

secondtonone1:这么优秀吗

asio实现http服务器

secondtonone1:我认为IO复用节省了线程资源，不用像传统服务器一样为每个连接单独创立线程，而是统一通过epoll或者iocp通知。

类和对象

陈宇航:支持！！！！

解决博客回复区被脚本注入的问题

secondtonone1:走到现在我忽然明白一个道理，无论工作也好生活也罢，最重要的是开心，即使一份安稳的工作不能给我带来事业上的积累也要合理的舍弃，所以我还是想去做喜欢的方向。

泛型算法的定制操作

secondtonone1:lambda和bind是C11新增的利器，善于利用这两个机制可以极大地提升编程安全性和效率。

双链表实现LRU算法

secondtonone1:双链表插入和删除节点是本篇的难点，多多练习即可。

构造函数

secondtonone1:构造函数是类的基础知识，要着重掌握

面试题汇总(一)

secondtonone1:看到网络上经常提问的go的问题，做了一下汇总，结合自己的经验给出的答案，如有纰漏，望指正批评。

使用mongo

secondtonone1:mongo操作是程序员必备的基础技能，包括增删改查，以及较为复杂的查询，分组查询，条件查询，模糊查询，文档内查询等，本文结合自己实际开发经验做了整理。

interface应用

secondtonone1:interface是万能类型，但是使用时要转换为实际类型来使用。interface丰富了go的多态特性，也降低了传统面向对象语言的耦合性。

golang 函数介绍

secondtonone1:函数是go中的一等公民，作为新兴语言，go摒弃了面向对象的一些糟粕，采取接口方式编程，而接口方式编程都是基于函数的，参数为interface，进而达到泛型作用，比如sort排序，只需要传入的参数满…

堆排序

secondtonone1:堆排序非常实用，定时器就是这个原理制作的。

Linux环境搭建和编码

恋恋风辰:Linux环境下go的安装比较简单，可以不用设置GOPATH环境变量，后期我们学习go mod 之后就拜托了go文件目录的限制了。

slice介绍和使用

恋恋风辰:切片作为引用类型极大的提高了数据传递的效率和性能，但也要注意切片的浅拷贝隐患，算是一把双刃剑，这世间的常态就是在两极之间寻求一种稳定。

其他链接

GitHub (https://github.com)
个人博客 (https://www.limerence2017.com)
博客园 (https://www.cnblogs.com/secondtonone1/)
CSDN (https://blog.csdn.net/secondtonone1)

B站



个人微信



恋恋风辰

北京 朝阳



扫一扫上面的二维码图案，加我为朋友。

个人公众号



本站累计访问量204866次

本网站内容全部为原创，谢绝转载。

京公网安备 11010502047625号 (http://www.beian.gov.cn/portal/registerSystemInfo?recordcode=11010502047625)

京ICP备2021033081号 (https://beian.miit.gov.cn/)